

PRACTICAL EXPERIMENT – 4

Name: K ANSHU

Roll No: 2520030440

Section: 8

Skill: Lexical Analysis, Tokenization, Command Parsing and Syntax Validation

Title

Implementation of Tokenization and Parser Logic for Command Input.

Aim

To split command input into tokens, identify delimiters, handle whitespace, create token structures, validate token streams, generate a parse or execution structure, and handle empty commands.

Theory

Command parsing begins by splitting input into individual tokens using delimiters such as spaces and tabs. The tokens are stored in a token structure and validated before an execution structure is produced. The parser also handles empty commands and identifies invalid token sequences.

Algorithm

1. Start the program.
2. Read a command from the user.
3. Check whether the command is empty.
4. Split the input into tokens using whitespace delimiters.
5. Store each token in the token structure.
6. Validate the token stream.
7. Detect invalid token sequences.
8. Generate the parse or execution structure.
9. Display the tokens and parsing result.
10. Repeat until the exit command is entered.
11. Stop the program.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50

typedef struct
{
    char *value;
} Token;

typedef struct
{
    Token tokens[MAX_TOKENS];
```

```

    int count;
} TokenStream;

void tokenize(char *input, TokenStream *stream)
{
    stream->count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && stream->count < MAX_TOKENS)
    {
        stream->tokens[stream->count].value = token;
        stream->count++;

        token = strtok(NULL, " \t\n");
    }
}

int validate_tokens(TokenStream *stream)
{
    if (stream->count == 0)
        return 0;

    if (strcmp(stream->tokens[0].value, "|") == 0)
        return -1;

    if (strcmp(stream->tokens[stream->count - 1].value, "|") == 0)
        return -1;

    for (int i = 0; i < stream->count - 1; i++)
    {
        if (strcmp(stream->tokens[i].value, "|") == 0 &&
            strcmp(stream->tokens[i + 1].value, "|") == 0)
        {
            return -1;
        }
    }

    return 1;
}

void display_parse_tree(TokenStream *stream)
{
    printf("\nParse / Execution Structure:\n");
    printf("COMMAND\n");

    for (int i = 0; i < stream->count; i++)
    {
        if (strcmp(stream->tokens[i].value, "|") == 0)
        {
            printf("  PIPE\n");
        }
        else
        {
            printf("  ARGUMENT: %s\n", stream->tokens[i].value);
        }
    }
}

int main()
{
    char input[MAX_INPUT];
    TokenStream stream;

    printf("=== OSSP Experiment 4 ===\n");
    printf("Tokenizer and Parser\n");
    printf("Enter a command to parse.\n");

```

```

printf("Type exit to quit.\n\n");

while (1)
{
    printf("parser> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    if (strcmp(input, "\n") == 0)
    {
        printf("Empty command detected.\n");
        continue;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;

    tokenize(input, &stream);

    printf("\nTokens:\n");

    for (int i = 0; i < stream.count; i++)
    {
        printf("Token %d: %s\n",
            i + 1,
            stream.tokens[i].value);
    }

    int result = validate_tokens(&stream);

    if (result == -1)
    {
        printf("\nSyntax Error: Invalid token sequence.\n");
        continue;
    }

    display_parse_tree(&stream);
    printf("\nParsing successful.\n");
}

printf("Exiting parser...\n");

return 0;
}

```

Execution Screenshots

1. Main C Code – Part 1

```

admin@heeraaj@DESKTOP-J39GHP7:~$ cat > main.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50

typedef struct
{
    char *value;
} Token;

typedef struct
{
    Token tokens[MAX_TOKENS];
    int count;
} TokenStream;

void tokenize(char *input, TokenStream *stream)
{
    stream->count = 0;

    char *token = strtok(input, " \t\n");

    while (token != NULL && stream->count < MAX_TOKENS)
    {
        stream->tokens[stream->count].value = token;
        stream->count++;

        token = strtok(NULL, " \t\n");
    }
}

int validate_tokens(TokenStream *stream)
{
    if (stream->count == 0)
        return 0;

    if (strcmp(stream->tokens[0].value, "|") == 0)
        return -1;

    if (strcmp(stream->tokens[stream->count - 1].value, "|") == 0)
        return -1;
}

```

2. Main C Code – Part 2

```

void display_parse_tree(TokenStream *stream)
{
    printf("\nParse / Execution Structure:\n");
    printf("COMMAND\n");

    for (int i = 0; i < stream->count; i++)
    {
        if (strcmp(stream->tokens[i].value, "|") == 0)
        {
            printf(" PIPE\n");
        }
        else
        {
            printf(" ARGUMENT: %s\n", stream->tokens[i].value);
        }
    }
}

int main()
{
    char input[MAX_INPUT];
    TokenStream stream;

    printf("=== OSSP Experiment 4 ===\n");
    printf("Tokenizer and Parser\n");
    printf("Enter a command to parse.\n");
    printf("Type exit to quit.\n\n");

    while (1)
    {
        printf("parser> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (strcmp(input, "\n") == 0)
        {
            printf("Empty command detected.\n");
            continue;
        }

        input[strcspn(input, "\n")] = '\0';
    }
}

```

3. Main C Code – Part 3

```

        if (result == -1)
        {
            printf("\nSyntax Error: Invalid token sequence.\n");
            continue;
        }

        display_parse_tree(&stream);
        printf("\nParsing successful.\n");
    }

    printf("Exiting parser...\n");

    return 0;
}

```

4. Compilation and Tokenization / Parsing

```
gcc main.c -o ossp
./ossp
=== OSSP Experiment 4 ===
Tokenizer and Parser
Enter a command to parse.
Type exit to quit.
```

```
parser> ls -l /home
```

Tokens:

Token 1: ls

Token 2: -l

Token 3: /home

Parse / Execution Structure:

COMMAND

ARGUMENT: ls

ARGUMENT: -l

ARGUMENT: /home

Parsing successful.

```
parser> ls -l | grep test
```

Tokens:

Token 1: ls

Token 2: -l

Token 3: |

Token 4: grep

Token 5: test

Parse / Execution Structure:

COMMAND

ARGUMENT: ls

ARGUMENT: -l

5. Empty Command Handling

```
Tokens:
Token 1: ls
Token 2: ||
Token 3: grep

Parse / Execution Structure:
COMMAND
  ARGUMENT: ls
  ARGUMENT: ||
  ARGUMENT: grep

Parsing successful.
parser>
Empty command detected.
parser> |
```

Result

The tokenizer and parser program was executed successfully. Command input was split into tokens, token structures and parse/execution structures were displayed, and empty command input was detected and handled.